

レポート課題 9

05-191009 奥村真司

問 1

動作例

```
# (1,2);;
for all
  ( int,  int)
- = (1,2)
# (fun x -> x,fun x -> x+1);;
for all a1
  ( ( a1 ->  a1 ),  ( int->  int))
- = (<fun>,<fun>)
-----
# fun x -> fun y -> match (x,y) with
                        | p -> (fun p -> match p with | (x,y) -> x) p;;
for all a6 a5
  ( a5 ->  ( a6 ->  a5 ))
-----
# fun x -> fun y -> fun z -> match ((x,y),z) with
  | tup -> (fun p -> match p with | (x,y) -> x) tup;;
for all a2 a7 a1
  ( a1 ->  ( a2 ->  ( a7 ->  ( a1 ,  a2 ))))
- = <fun>
-----
# [];;
for all a3
  a3  list
- = []
# 1::[];;
for all
  int list
- = (1::[])
# (1,true);;
for all
  ( int,  bool )
- = (1,true)
# 2::1::[];;
for all
  int list
- = (2::(1::[]))
```

考察

`eval_expr` で `EPair(x,y)` ならば `x,y` を評価して `(x,y)` を返し、`ECons(x,y)` ならば `x,y` を評価して、`x::y` を返した。

問 2

動作例

```
# match (1,2) with
| (a,b) -> a+b;;
for all
  int
- = 3
# let f = fun x -> fun y ->
  match (x,y) with
  | (1,a) -> a
  | (b,c) -> b*c;;
for all
  ( int-> ( int-> int))
val f = <fun>
# f 1 5;;
for all
  int
- = 5
# f 2 5;;
for all
  int
- = 10
# f 2 3;;
for all
  int
- = 6
# let rec f x1 =
  match x1 with
  | [] -> 0
  | x::xs -> x+(f xs);;
for all
  ( int list-> int)
val f = <rec fun>
# f (1::2::3::[]);;
for all
  int
- = 6
```

考察

例題で作成した `find_match` は `value` が `pattern` にマッチしないとき `PatternMatchError` という例外を投げるようにつくった。パターンを 1 つめから順に `PatternMatchError` を `try-with` 文で補足しながら試していき、初めてマッチしたときのパターンに対する割り当てで環境を拡張し、その元でパターンに対応する式を評価した。

問 3

動作例

```
# 1::true::[];;
Fatal error: exception TySyntax.TyError
-----
# (1,true);;
for all
  ( int,  bool )
- = (1,true)
# (fun x->x,fun x->x);;
for all a2 a1
  ( ( a1 ->  a1 ),  ( a2 ->  a2 ))
- = (<fun>,<fun>)
# match fun x->x with
  | f -> (f 1,f true);;
Fatal error: exception TySyntax.TyError
```

考察

スライドの方針を参考に実装した。(問 1,2 の動作例の出力も問 3 まで実装したあとのものなので型推論が行われているのでそちらも参照していただければと思います。) 用意した補助関数の一部を説明する。`match e with p1 -> e1 | ... | pn -> en` の各 p に対して型, 制約、追加される型環境 (t, c, Γ) を求めているのが `tcgamma,  $\Gamma_i$` を元の型環境に加えて e_i を評価するのが `add_g_infer`、型 t と型のリスト $t_1, t_2, t_3, \dots$ を受け取って $t = t_1 = t_2 = t_3 \dots$ の制約を生成するのが `make_cons` である。

スライドの方針では最後の動作例のように `match-with` の間のものが多相関数にならない。この動作は本家 Ocaml インタプリタと異なり気になったので発展 2 にて多相関数へ対応できるように拡張した。

問 4

動作例

```
# (fun x -> (x*x*x+x)-(x*x)) (1+2);;
for all
  int
EApp (EFun (x,ESub (EAdd (EMul (EMul (x,x),x),x),EMul (x,x))),EAdd (1,2))
EFun (x,ESub (EAdd (EMul (EMul (x,x),x),x),EMul (x,x)))
ESub (EAdd (EMul (EMul (x,x),x),x),EMul (x,x))
EAdd (EMul (EMul (x,x),x),x)
EMul (EMul (x,x),x)
EMul (x,x)
x
EAdd (1,2)
1
2
x
```

```

EAdd (1,2)
1
2
x
EAdd (1,2)
1
2
x
EAdd (1,2)
1
2
EMul (x,x)
x
EAdd (1,2)
1
2
x
EAdd (1,2)
1
2
- = 21
-----
# let rec f n = if n = 0 then 0 else n + f ( n - 1 );;
for all
  ( int-> int )
val f = <rec fun>
# f 10000;;
( about 10 seconds later... )
for all
  int
- = 50005000

```

考察

環境を変数とサンクの組に変更し、eval_expr 内でいままで行っていた「環境に値を追加してから評価」という部分を適宜サンクを追加するように置き換えた。動作例は eval_expr 内で評価前に引数の式を表示するようにしたもので、動作例を見ると (1+2) が何度も評価されていることがわかる。点線の下の方の例の f は 1 から n までの和を求める再帰関数である。この計算量について考える。今までのインタプリタではこの計算量は $O(n)$ である。以下で、名前呼びでの f の計算量が少なくとも $O(n^2)$ であることを示す。名前呼びであるから、f 3 を呼ぶと中で f (3-1) が呼ばれ、f (3-1) の中で f ((3-1)-1) が呼ばれ... というように呼ばれる。f k では k が else の部分で評価されるが、k の評価には n-k に比例する時間がかかる。よって全体では $1 + 2 + 3... + n$ に比例する時間以上かかるので $O(n^2)$ 以上かかる。よって f 10000 を評価するのに 10 秒もかかる。

発展 1

動作例

```

# (fun x -> (x*x*x+x)-(x*x)) (1+2);;
for all
  int

```

```

EApp (EFun (x,ESub (EAdd (EMul (EMul (x,x),x),x),EMul (x,x))),EAdd (1,2))
EFun (x,ESub (EAdd (EMul (EMul (x,x),x),x),EMul (x,x)))
ESub (EAdd (EMul (EMul (x,x),x),x),EMul (x,x))
EAdd (EMul (EMul (x,x),x),x)
EMul (EMul (x,x),x)
EMul (x,x)
x
EAdd (1,2)
1
2
x
x
x
EMul (x,x)
x
x
- = 21
-----
# let rec f n = if n = 0 then 0 else n + f ( n - 1);;
for all
  ( int-> int)
val f = <rec fun>
# f 10000;;
for all
  int
- = 50005000 (in a flash)

```

考察

問4でサンクを環境に追加していたところを適宜 `ref DThunk` に置き換え、`EVar` の評価の際にもし参照先が `DThunk` なら評価して値に変えるようにした。

上の動作例を見ると、`1+2` の評価は一度しか行われていないことがわかる。必要呼びでは値呼びと同様に、名前呼びのときのような不要な再計算がおこらないので動作例の `f` の計算量は $O(n)$ であり、高速である。

発展 2

動作例

```

# match fun x ->x with
  | f -> (f 1,f true);;
for all
  ( int, bool )
- = (1,true)
# match fun x -> fun y -> x+1 with
  | f -> (f 1 true,f 2 false);;
for all
  ( int, int)
- = (2,3)
# match fun x -> let g = fun y -> x y in g 4 with
  | f -> (f (fun x -> x+1),f (fun x -> true));;
for all
  ( int, bool )

```

```
- = (5,true)
# match (match fun x -> fun y -> fun z -> z with | f -> (f 1,f true)) with
  | (g,h) -> ((g 2 3),(h true []));;
for all a23
  ( int,  a23  list)
- = (3,[])
-----
# ->
Error! /* <- color */
Parsing.Parse_error
```

考察

2つの拡張を行った。

パターンマッチの let 多相への対応

パターンマッチの型推論の実装を授業スライドの方針に従ってやると、上の動作例にあるような

```
# match fun x ->x with
  | f -> (f 1,f true);;
```

という式が型エラーとなるが、本家 Ocaml インタプリタではこれは

```
# let f = fun x ->x in (f 1,f true);;
```

と同様に多相関数となり型エラーにはならないことを同期に教えてもらったので、本家の仕様に合わせるべく型推論器の拡張を行った。まず、授業スライドの方針でなぜ型エラーになるかについて考えた。上の例で、パターンの f の型はとりあえず新たな型変数 α と置かれる。このまま unification が行われるため f が型スキーマに一般化されることがなく多相にならない。なので let 多相でやったようにどこかしらで一旦 unify し、それで未決定なものを generalize してやる必要がある。授業スライドではパターンの型全てと match-with で挟まれた式の型が等しいという制約と、ARROW の右側の型が全て同じという制約を同時に unify しているがこれらは独立であることに着目した。なのでパターンの型全てと match-with で挟まれた式の型が等しいという制約 (leftC とする) をまず集めて、unify しその結果の subst で置き換えてもなお未決定な型変数は generalize で型スキーマに一般化する。そのもとで ARROW の右側の型が全て同じという制約 (rightC とする) を集め、unify すればよい。スライドでは追加する型環境 (= 変数名と型スキーマの組) Γ を求めているが、代わりに追加する変数名と型の組 Γ を求めた。leftC を unify し、それを sigma をとし、あとは let 多相のときと同様に型環境の置き換えや generalize などを行った。結果的に動作例のように match-with 文の let 多相への対応ができた。

エラー処理と表示

エラーが起こってもインタプリタが終了しないように try-with 文を用いてエラーを捕捉するようになった。また Error をカラーで表示するようにした。